

Training a Code LLM from Scratch • Part 4

RL for Code Models: from Policy Gradient to GRPO, from Single-Step Generation to Multi-Step Debug

First-principles derivation of RL for code generation (single-step bandit) and code debug / SWE-agent (multi-step MDP). Includes the full $J(\theta)$ expansion, law of large numbers, log-derivative trick, GRPO, advantage-weighted self-generated-sequence CE, and LoRA. Foundation for the Kaggle SWE-agent competition.

Outline

- 1. The core of RL: one sentence once you strip the math packaging
- 2. The full expansion of $J(\theta)$ (policy prob, transition, V)
- 3. Policy gradient: log-derivative trick + law of large numbers (Monte Carlo)
- 4. Advantage: $A=Q-V$, degenerating to r -baseline at single step
- 5. Code generation: single-step bandit + GRPO (drop critic)
- 6. policy loss = advantage-weighted CE over the self-generated sequence
- 7. clip and KL anchoring (PPO's proximal)
- 8. Framework vs algorithm: RLVR / RLHF / GRPO / PPO
- 9. Code debug / SWE-agent: multi-step MDP + GAE
- 10. State representation: text / execution graph / variable values, and the LLM's "mental execution simulation"
- 11. LoRA: freeze base, train only the low-rank delta

1. The core of RL: strip the math packaging

Textbooks make RL winding (MDP, Bellman, policy-gradient theorem, importance sampling, GAE...), but the core is one sentence: **sample a batch of actions → compute how much better than average each is (advantage) → raise the probability of good ones, lower bad ones (advantage-weighted log-probability gradient)**. Make high-return actions more likely. The jargon is packaging; the mechanism is minimal.

2. The full expansion of $J(\theta)$

RL's objective: **maximize the expected return of the behavior produced by policy π (params θ)**. Training = adjust θ to maximize J = gradient ascent $\theta \leftarrow \theta + \text{lr} \cdot \nabla J$. The core task is **computing ∇J** — but J is an expectation (over actions sampled by the policy), and the gradient of an expectation isn't directly computable; this is what policy gradient solves.

- $V(s)$ = expected total return from state s following $\pi = E[\sum \gamma^t r_t \mid s_0=s]$.
- $Q(s,a)$ = expected total return taking a in s then following π .
- Bellman: $Q(s,a) = r(s,a) + \gamma \sum_{s'} P(s'|s,a) V(s')$.
- $J(\theta) = V(s_0) = \sum_a \pi(a|s_0) Q(s_0,a)$.

$$J(\theta) = E_{a \sim \pi}[R] = \sum_a \pi(a|s_0) Q(s_0,a), \quad Q(s_0,a) = \sum_{s'} P(s'|s_0,a) V(s')$$

Back-and-forth (transition sparsity) — For many tasks (esp. LLM generation), given (s_0, a) the next state is essentially **deterministic** (the sequence with the generated token appended is unique) $\rightarrow$ the transition $P(s'|s_0,a)$ is one-hot $\rightarrow \sum_{s'}$ degenerates to "V of the one deterministic next state". So in LLMs the transition layer is very sparse (deterministic).

3. Policy gradient: log-derivative trick + law of large numbers

$J(\theta) = \sum_a \pi_\theta(a) R(a)$. Gradient:

$$\nabla_\theta J = \sum_a \nabla_\theta \pi_\theta(a) \cdot R(a)$$

Problem: $\nabla \pi$ isn't an expectation form (can't sample). Trick — multiply and divide by π :

$$\nabla_\theta J = \sum_a \pi_\theta(a) \cdot [\nabla \pi / \pi] \cdot R(a) = \sum_a \pi_\theta(a) \cdot \nabla_\theta \log \pi_\theta(a) \cdot R(a)$$

(using $\nabla \log \pi = \nabla \pi / \pi$). The right side is now " π -weighted sum" = **expectation**:

$$\nabla_\theta J = E_{a \sim \pi}[R(a) \cdot \nabla_\theta \log \pi_\theta(a)].$$

Why the trick matters: $\nabla \pi$ can't be sampled (not in " π -weighted" form), but $\nabla \log \pi$ times π turns back into a " π -weighted expectation" $\rightarrow$ samplable. **This is the core math of policy gradient** — turning "gradient of a probability" into "expectation of the samplable log-probability gradient".

Law of large numbers is the foundation (textbooks skip it): $\nabla J = E[\dots]$ is an expectation whose true value requires integrating over all actions/trajectories (uncomputable). LLN: sampling N iid $a_i \sim \pi$, the sample mean $(1/N) \sum R(a_i) \nabla \log \pi(a_i)$ **converges in probability to the true expectation** ($N \rightarrow \infty$). So $\nabla J \approx (1/N) \sum R(a_i) \nabla \log \pi(a_i)$. **Both expectations rely on sampling:** ① outer over actions (sample N solutions/trajectories) ② inner R/V is itself an expectation (sampled trajectory returns approximate V 's definition).

This is Monte Carlo estimation; textbooks gloss it (" $E \approx (1/N) \sum$ ") without noting the foundation is LLN. Consequences: ① N must be large (few samples $\rightarrow$ high variance) ② RL is inherently high-variance and hard to train (sampling noise) ③ subtracting a baseline, GAE, etc. are all **variance-reduction** tricks within the sampling-estimation framework.

4. Advantage: $A = Q - V$

Weighting by R directly has high variance. Subtract a baseline b (action-independent): $\nabla J = E[(R-b) \cdot \nabla \log \pi]$. **Key: subtracting b doesn't change the expectation** ($E[b \cdot \nabla \log \pi] = b \cdot \nabla (\sum \pi) = b \cdot \nabla 1 = 0$), but **reduces variance**. Take $b = V(s) \rightarrow R-V \approx Q-V = \text{advantage}$. **So advantage = return with baseline subtracted**: same expectation, lower variance. $A > 0$ reinforce, $A < 0$ suppress.

Estimating Q : don't estimate Q separately; use $Q(s,a) = r + \gamma V(s')$ (Bellman). So $A = r + \gamma V(s') - V(s)$ (TD advantage). **Only need critic for V** (one net); Q borrows $V(s')$; $V(s')$ still needs critic. **GAE** weights multiple n -step TDs, trading off bias (relying on inaccurate V) and variance (sampling noise) — an estimate, not exact (true V unknown, critic only approximates). This is the GAE you hand-wrote in CarRacing.

Back-and-forth ($V(s)$ at single step) — At single step $V(s)$ isn't meaningful, it **degenerates to baseline (mean reward)**: single step takes a , gets r (no s'), $Q=r$, $V(s)=\text{expectation of all actions' rewards} = \text{mean} \rightarrow A = r - \text{mean reward}$.

GAE bias-variance: one-step TD ($A=r+\gamma V(s')-V(s)$, low variance high bias) / actual multi-step return (low bias high variance) / GAE (λ -weighted n -steps, $A=\sum(\gamma\lambda)^t \delta_t$, trade-off). All estimates, not exact.

5. Code generation: single-step bandit + GRPO

Modeling: give problem $\rightarrow$ generate solution $\rightarrow$ test $\rightarrow$ reward $\rightarrow$ **end** (no state transition). **Single-step contextual bandit**: no state transition (state = the problem, fixed) $\rightarrow$ no multi-step MDP needed. action = generated solution (token sequence); **sample N solutions at different temperatures** (diversity from temperature spread: low-temp conservative + high-temp exploratory). reward = **partial (fraction of tests passed, e.g. 8/10)**, denser than 0/1 — GRPO's in-group comparison can distinguish "passed 8 vs 5" (both 0 under 0/1, incomparable).

GRPO (Group Relative Policy Optimization, drop critic): single step, no s' , $A = r - \text{baseline}$. **baseline not from a critic net, from the group's N solutions' mean reward**: $A_i = (r_i - \text{mean}(r)) / \text{std}(r)$ (group normalization; $\div \text{std}$ makes scales comparable, training stabler). **Group mean as baseline = dropped PPO's critic/value net**. This is GRPO's core simplification over PPO.

Back-and-forth (three modeling corrections) — ① temperature is a **sampling hyperparameter** (means to get different solutions), not an action (the solution is the action); ② pass/fail is the

reward (outcome), not state (state is the problem); ③ code generation is a **single-step bandit**, **no state concept needed** (problem fixed) — multi-step state is for debug/SWE-agent.

Why single step needs no critic: multi-step $A=r+\gamma V(s')-V(s)$ needs critic for V . Single step (bandit): ends after generation, no s' , $A=r$ -baseline; baseline just needs "mean reward" — GRPO uses group mean directly, no value net.

6. policy loss = advantage-weighted CE over the self-generated sequence

RL has no supervised label (no "correct solution"). policy loss: $L = - \sum_i A_i \log \pi(\text{sol}_i)$. "label" = **self-sampled solution**, "weight" = **advantage**. $A_i > 0$ raise this solution's prob, $A_i < 0$ lower. Derivative (A as constant): $\nabla L = - \sum A_i \nabla \log \pi(\text{sol}_i)$.

Two layers: outer A_i **for the i-th whole solution** (solution-level advantage); inner this solution is **still per-token CE** ($\log \pi(\text{sol}) = \sum_t \log P(\text{token}_t | \text{prefix})$, autoregressive, same as normal LLM training). Together: $A_i \times (\text{this solution's per-token CE}) = \text{solution-level advantage} \times \text{token-level CE}$.

CE vs RL (form alike, source different): CE (supervised): $-\log P(\text{external correct token})$, coefficient fixed 1. RL: $-A \cdot \log P(\text{self-generated token})$, coefficient = advantage. Same $\log P$ form, but RL's token is self-generated, coefficient is A . **RL = advantage-weighted CE over self-generated sequences. Gradient:** like CE, under softmax **all output logits backprop** (not just sampled), whole network all layers (embedding etc. all update).

Back-and-forth (V/critic's two roles) — V /critic has two independent roles: ① trained in **critic loss** (MSE, V toward actual return) ② a **constant (detached)** in **policy loss** (A contains V , but no gradient to A when updating policy). PPO has both losses; GRPO only policy loss (A from group mean, no critic loss).

Back-and-forth (all logits backprop) — Once thought RL "only backprops through the sampled action's logit". **Correction:** under softmax **all logits backprop** (like CE): sampled action's logit explicit, others implicit via softmax normalization. CE vs RL differ not in "which logits" (all), but in **target source** (external one-hot correct token vs self-sampled + advantage).

7. clip and KL anchoring (PPO's proximal)

- **clip(ratio):** replace $\log \pi$ with ratio $r = \pi_{\text{new}}/\pi_{\text{old}}$, clip to $[1-\epsilon, 1+\epsilon]$: $L = - \sum \min(r \cdot A, \text{clip}(r, 1-\epsilon, 1+\epsilon) \cdot A)$. **Limits new-old policy divergence** (Proximal) — prevents too-large steps; and

policy gradient estimates from π_{old} -sampled data, if π_{new} strays too far the estimate is off (importance sampling fails), clip keeps within the valid neighborhood.

- **KL anchoring:** $-\beta \cdot \text{KL}(\pi \parallel \pi_{\text{ref}})$, ref = SFT/initial model (e.g. Gemma-4-it). Prevents drift, prevents reward hacking, avoids forgetting original abilities.

8. Framework vs algorithm: RLVR / RLHF / GRPO / PPO

	what	difference
RLHF	framework	reward = human preference (learned RM, subjective, prone to reward hacking)
RLVR	framework	reward = verifiable correctness (tests/answers, objective, unhackable) — code uses this
PPO	algorithm	with critic (estimates V), clip proximal
GRPO	algorithm	drops critic (group mean as baseline), clip proximal

Framework (RLVR/RLHF) decides reward source; **algorithm (PPO/GRPO)** decides how to update. The underlying math is shared (single/multi-step policy gradient + advantage weighting), only reward modeling differs. **Your code RL = RLVR (tests as reward) + GRPO (drop critic)**. RLVR's reward is objective and trustworthy (unlike RLHF's noisy RM), the reason DeepSeek-R1 can emerge reasoning via pure RL.

9. Code debug / SWE-agent: multi-step MDP + GAE

The core change from code generation: **single-step bandit** → **multi-step MDP** (with state transition). state: current execution state (output/error) + current code + problem + history. action: generate the next edit based on state (error/output); SWE-agent also includes navigation (read files/search, the competition's 9 tools). transition: edit code → re-execute → new state (new output/error). reward: partial (degree of resolution / test pass fraction / error reduction). **Multi-step MDP** (with transition, not bandit) → use **GAE / cumulative return / credit assignment** (which step's edit was key).

Kaggle SWE-agent is essentially code debug: given repo + issue → navigate repo → localize → edit → run tests to verify (multi-turn: localize→edit→test→fix). Same class as this section's debug modeling. **Difference only in scale/tools:** SWE-agent is "big-repo debug" (adds navigation/read/search tools), same essence (multi-step debug loop). This modeling applies directly to Kaggle: state=repo/error, action=edit+navigate, reward=test partial, multi-step MDP (GAE).

Single-step → multi-step essence: code generation (single-step bandit): problem → one generation → reward → end, no transition, $V(s)$ degenerates to mean. debug (multi-step MDP): edit → execute → new error → edit again, **with transition** (code state changes per edit), back to full RL: cumulative return, $V(s)$ meaningful (expected return from current code state to

fixed), GAE for multi-step credit assignment. **This is exactly CarRacing's multi-step RL** (trajectory/GAE/credit assignment) — debug's multi-turn = CarRacing's multi-step, framework transfers directly, swap environment (racing → code-execution).

Credit assignment (multi-step difficulty): in multi-turn debug, when finally fixed, **which edit was key?** (maybe the 3rd fixed the core, first two explored). GAE does this with discount + TD (later near-success steps get more credit). Sparse reward (only final success) makes it harder → process reward (per-step partial improvement) helps.

10. State representation: text / execution graph / variable values

Encoding state matters (how to represent high-dim code/error decides agent decisions). Three layers (easy to hard): **text (easy)**: code, error text, unit-test failure info (line/expected vs actual) → embed directly (LLM native). Mainstream. **execution graph (embeddable)**: AST/call graph/dataflow → **serialize graph to text** (or code-graph embeddings, competition-provided). **variable values/execution internals (hardest)**: dynamic, unbounded, high-dim (arbitrary values). Predefined states (discretize) lose info; new states explode — no clean solution, in practice "key-variable text snapshot + truncation".

Back-and-forth (does Kaggle have variable-value debug) — Kaggle's 9 tools: code structure (code graph/neighbors/subgraph) + run_command (run tests, get error text), **no debugger/variable-value/single-step tools** — the action set is fixed, state relies on "code + error text", no "runtime variable values". Reasons: ① action set fixed ② execution-internal state modeling too hard (officially avoided). So the competition's SWE-agent is "code+error" debug (text state), not "variable-value" deep debug.

The LLM's "mental execution simulation" (emergent): strong LLMs (Claude/GPT) editing code will, in the reasoning chain, "assume input X, walk the code, variables become..., error here" — **but this is mental simulation (CoT reasoning), not real execution/trace**. The ability comes from pretraining + reasoning RL (seen massive "code→execution result" pairs; next-token forces learning the "code→behavior" generalizing map; R1-style CoT reinforces). Still next-token fitting a "world model of code execution" — generalizes to new programs. **How it's implemented internally: black box (uninterpretable)**, but the ability is real. Two different things from real trace (external variable values fed in): one is internal mental sim, one is external real execution fed in.

11. LoRA: freeze base, train only the low-rank delta

Don't change original W (freeze), add a low-rank increment ΔW to each matrix: $\Delta W = B \cdot A$ ($B: d \times r, A: r \times d$, small r like 16), forward = $X \cdot W + X \cdot (B \cdot A)$. **delta isn't as big as original W :** store only B, A (e.g. $8192 \times 16 + 16 \times 5376 \approx 220K$, $\sim 0.5\%$ of W 's 44M). Gradient only for $B, A \rightarrow$

AdamW state only for that 0.5% → saves memory (QLoRA fits one card). Assumption: the ΔW needed for fine-tuning is low-rank (a few directions suffice). **SFT and RL both train only LoRA** (base frozen). Matches competition rules (base official-fixed, only LoRA adapter allowed) + cheaper.

Why low-rank suffices: fine-tuning (adapting to a task) needs a ΔW of low intrinsic dimension — not a full-rank rewrite, a few directions suffice (adapt $\neq$ relearn ability). This is why strong-base + LoRA works (ability in base, LoRA just low-rank alignment); while weak base (small from-scratch model) can't be saved by LoRA (ability absent, not something low-rank fine-tuning fills).

QLoRA memory account: full-param training's memory hog is AdamW state (m, v FP32, $8 \times$ params). QLoRA: 4bit-quantized frozen base (no grad/optimizer) + only LoRA ($\sim 0.5\%$) has grad and AdamW → compresses the " $\times 8$ AdamW hog" to just 0.5%. 31B full $\sim 400G+$, QLoRA $\sim 40-55G$, one card.

Summary

The full chain of RL for code models: $J(\theta) = E[R] \rightarrow$ **log-derivative trick** ($\nabla P = P \nabla \log P$) + **LLN** (Monte Carlo sampling estimates expectations) $\rightarrow$ **advantage** ($A = Q - V$ variance reduction) $\rightarrow$ **code generation** (single-step bandit + GRPO group baseline drops critic) $\rightarrow$ **policy loss** (advantage-weighted CE over self-generated sequences, outer solution-level $A \times$ inner token CE) $\rightarrow$ **clip + KL anchoring** (proximal) $\rightarrow$ **debug/SWE-agent** (multi-step MDP + GAE) $\rightarrow$ **state representation** (text/graph/variable values) $\rightarrow$ **train only LoRA** (base frozen). Framework (RLVR) + algorithm (GRPO) + LoRA + multi-step (SWE-agent) — exactly the full theoretical foundation for the next Kaggle Gemma-4 SWE-agent competition.

— End of Part 4. Theory complete; next: Kaggle SWE-agent competition engineering. —